

FULL STACK DEVELOPMENT

FASE 5 | AULA 03 -

PRINCIPAIS DIFERENÇAS AO DESENVOLVER MICROSERVIÇOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	17
O QUE VOCÊ VIU NESTA AULA?	20
REFERÊNCIAS	21

EMANDA

O QUE VEM POR AÍ?

Nesta aula, exploraremos as ferramentas e tecnologias essenciais para o desenvolvimento de microserviços, incluindo linguagens de programação e frameworks populares, ferramentas de containerização e orquestração, e práticas recomendadas de desenvolvimento, teste e deploy contínuo. Também vamos analisar as diferenças entre o desenvolvimento de microserviços e monolitos, destacando os desafios e benefícios de cada abordagem.



HANDS ON

Nesta aula prática, serão apresentadas as ferramentas e tecnologias utilizadas no desenvolvimento de microserviços, com foco em linguagens de programação, frameworks, containerização e orquestração. Vamos explorar práticas recomendadas para desenvolvimento, teste e deploy contínuo, bem como técnicas de monitoramento e logging. Te orientaremos para que possa implementar uma pequena aplicação utilizando essas ferramentas e tecnologias, comparando a experiência de desenvolvimento entre microserviços e monolitos.



SAIBA MAIS

Linguagens de programação e frameworks populares

Desenvolver microsserviços envolve a escolha de linguagens de programação e frameworks que melhor se adequem ao contexto e às necessidades específicas do projeto. Diferentemente das aplicações monolíticas, onde uma única linguagem é frequentemente utilizada para toda a base de código, os microsserviços permitem (e muitas vezes incentivam) o uso de múltiplas linguagens de programação. Cada serviço pode ser escrito na linguagem mais adequada às suas funções específicas, sem que isso comprometa a interoperabilidade entre os serviços.

Fatores a considerar na escolha da linguagem

A escolha da linguagem deve alinhar-se com os requisitos técnicos do serviço específico, ou seja, com as **necessidades do projeto**. Por exemplo, se um serviço exige alto desempenho e baixa latência, uma linguagem como Go ou Rust pode ser preferível. Por outro lado, se a prioridade for a rapidez no desenvolvimento e a facilidade de manutenção, linguagens como Python ou JavaScript podem ser mais adequadas.

A **familiaridade e a expertise da equipe** de desenvolvimento com uma determinada linguagem ou framework são cruciais. Utilizar uma linguagem que a equipe já domina pode acelerar o desenvolvimento e reduzir a curva de aprendizado. No entanto, não deve ser o único fator, pois é importante também considerar os requisitos técnicos e as características da linguagem.

A **disponibilidade de frameworks, bibliotecas e ferramentas** que suportam a linguagem pode influenciar a escolha. Um ecossistema rico pode facilitar a implementação de funcionalidades, integração com outras tecnologias e resolução de problemas comuns.

Interoperabilidade e comunicações é um fator muito importante na escolha da linguagem, pois em um ambiente de microsserviços, a comunicação entre serviços é geralmente realizada através de APIs bem definidas (REST, GraphQL, gRPC) ou mensagens. Isso significa que os serviços podem ser escritos em diferentes

linguagens, contanto que possam se comunicar de maneira eficaz. É essencial garantir que a linguagem escolhida suporte os padrões de comunicação adotados pelo projeto.

Linguagens populares e seus contextos

Java

Framework: Spring Boot.

Contexto: Java é amplamente utilizado em grandes empresas devido à sua robustez, desempenho e vasta biblioteca de frameworks e ferramentas. Spring Boot facilita a criação de aplicações Java autônomas e de produção prontas para execução, com suporte integrado para microserviços.

JavaScript/Node.js

Framework: Express.

Contexto: Node.js é conhecido por sua alta performance em operações de I/O e sua natureza assíncrona. É ideal para aplicações em tempo real e APIs leves, oferecendo um ecossistema vasto e uma curva de aprendizado suave para desenvolvedores(as) web.

Python

Frameworks: Flask e FastAPI.

Contexto: Python é valorizado por sua simplicidade e rapidez no desenvolvimento. É uma excelente escolha para serviços que requerem desenvolvimento ágil e que se beneficiam da vasta gama de bibliotecas e ferramentas de ciência de dados e machine learning.

Go (Golang)

Contexto: Go é reconhecido por sua alta performance e eficiência em processamento concorrente. É frequentemente utilizado para serviços que requerem alta velocidade e baixa latência, como servidores de alta demanda e sistemas de processamento de dados em tempo real.

C#

Framework: .NET Core.

Contexto: C# com .NET Core é ideal para empresas que já utilizam o ecossistema Microsoft. Oferece boa performance, suporte multiplataforma e uma vasta biblioteca de ferramentas de desenvolvimento.

O que procurar em uma linguagem para microserviços?

Linguagens que oferecem **bom suporte a processamento concorrente e paralelismo** são vantajosas para microserviços que necessitam lidar com muitas operações simultâneas.

Facilidade de integração com ferramentas DevOps também é um ponto importante, pois se a linguagem for compatível com ferramentas de CI/CD, monitoramento, logging e orquestração, a implementação e manutenção contínua dos serviços é facilitada.

A linguagem deve suportar os padrões de comunicação adotados (REST, GraphQL, gRPC), tendo **eficácia na comunicação entre serviços** e garantindo a interoperabilidade.

Um **ecossistema rico em bibliotecas e frameworks** pode acelerar o desenvolvimento e fornecer soluções para problemas comuns, reduzindo o tempo e o esforço necessários para implementar funcionalidades complexas.

Flexibilidade na escolha da linguagem

Uma das grandes vantagens dos microserviços é a flexibilidade para escolher a linguagem mais adequada para cada serviço, potencializando a eficiência e a qualidade do desenvolvimento. Por exemplo, uma aplicação de e-commerce pode usar Python para seu serviço de recomendação de produtos devido à sua forte integração com bibliotecas de machine learning, enquanto utiliza Go para serviços de processamento de pagamentos que requerem alta performance.

Exemplo prático de diversidade de linguagens

Considere uma aplicação de gerenciamento de saúde com os seguintes microsserviços:

1. Serviço de agendamento de consultas
 - Linguagem: Java com Spring Boot.
 - Motivo: robustez e facilidade de integração com sistemas legados de gestão hospitalar.
2. Serviço de notificações
 - Linguagem: Node.js com Express.
 - Motivo: alta performance em I/O e necessidade de integração rápida com serviços de envio de SMS e e-mails.
3. Serviço de análise de dados de pacientes
 - Linguagem: Python com FastAPI.
 - Motivo: utilização de bibliotecas de machine learning para análise preditiva de saúde.
4. Serviço de autenticação
 - Linguagem: Go.
 - Motivo: necessidade de alta performance e baixa latência para validar milhões de requisições de autenticação por segundo.

Ferramentas de containerização e orquestração

A containerização e a orquestração são fundamentais para o desenvolvimento e implantação de microsserviços. Elas permitem a consistência no ambiente de execução, a escalabilidade e a gestão eficiente de recursos, promovendo a agilidade e a robustez necessárias para ambientes de microsserviços. A seguir exploraremos as ferramentas mais populares e seus benefícios.

Containerização: Docker

Docker é a ferramenta de containerização mais utilizada. Ela permite empacotar uma aplicação e todas as suas dependências em um contêiner, que pode ser executado de forma consistente em qualquer ambiente. As suas principais características são:

Imagens e contêineres: Docker utiliza imagens para definir o conteúdo dos contêineres. Neste caso, uma imagem é um snapshot imutável de um ambiente de execução que inclui a aplicação, suas dependências e as configurações necessárias.

Portabilidade: um contêiner Docker pode ser executado em qualquer máquina que tenha o Docker instalado, garantindo que a aplicação funcione da mesma forma em diferentes ambientes (desenvolvimento, teste, produção).

Isolamento: os contêineres oferecem um ambiente isolado para a aplicação, reduzindo conflitos entre diferentes serviços e facilitando a gestão de dependências.

Alguns dos benefícios do Docker incluem:

Consistência: garantia de que o código funciona da mesma maneira em qualquer ambiente, eliminando o problema de "funcionava na minha máquina".

Rapidez no deploy: facilita e acelera o processo de deploy, permitindo que novos contêineres sejam instanciados rapidamente.

Escalabilidade: contêineres podem ser escalados facilmente, criando instâncias de serviços conforme a demanda.

Uma pessoa profissional em desenvolvimento pode criar uma imagem Docker para um serviço web, especificando todas as dependências e configurações necessárias em um arquivo **Dockerfile**. Esta imagem pode ser construída e executada como um contêiner em qualquer ambiente que suporte Docker.

Orquestração: Kubernetes

Kubernetes é a plataforma de orquestração de contêineres mais popular, usada para automatizar a implantação, o dimensionamento e a operação de aplicações containerizadas. A seguir veremos suas principais características:

Principais diferenças ao desenvolver microsserviços

Pods: Kubernetes agrupa contêineres em unidades chamadas Pods. Um Pod pode conter um ou mais contêineres que compartilham recursos de rede e armazenamento.

Deployment: define como as aplicações são implantadas, atualizadas e escaladas. Um Deployment garante que um número desejado de Pods esteja em execução.

Serviços: Kubernetes usa serviços para expor Pods para a rede interna ou externa, permitindo que os contêineres se comuniquem entre si e com o mundo externo.

ConfigMaps e secrets: gerenciam a configuração de aplicações e dados sensíveis como senhas e chaves API, separando-os do código da aplicação.

Volumes: permitem que os contêineres armazenem e compartilhem dados persistentes.

Alguns dos benefícios do Kubernetes são:

Automação de deployment: Kubernetes gerencia o ciclo de vida dos contêineres, automatizando tarefas como reiniciar contêineres falhos, escalar contêineres com base na demanda e distribuir cargas de trabalho de maneira eficiente.

Auto-escalabilidade: suporte para escalabilidade automática com base em métricas de uso de CPU e memória, garantindo que as aplicações possam lidar com variações de carga.

Resiliência: Kubernetes detecta e substitui contêineres que falham, garantindo alta disponibilidade das aplicações.

Pensando em um cenário real, um(a) administrador(a) pode usar Kubernetes para definir um Deployment que especifica a imagem Docker a ser usada, o número de réplicas (Pods) desejado e as políticas de atualização. Kubernetes gerencia automaticamente a criação, atualização e escalabilidade desses Pods.

Docker Compose

Docker Compose é uma ferramenta para definir e executar aplicações multi-contêiner. Ele utiliza um arquivo YAML para configurar os serviços de uma aplicação,

Principais diferenças ao desenvolver microsserviços

facilitando o desenvolvimento e teste local. Vejamos algumas de suas principais características:

Definição de serviços: permite definir múltiplos serviços em um único arquivo `docker-compose.yml`, especificando as imagens, volumes e redes necessários.

Ambientes de desenvolvimento local: ideal para configurar ambientes de desenvolvimento e teste que envolvem vários contêineres.

Orquestração Simples: permite iniciar, parar e gerenciar todos os serviços definidos com um único comando (`docker-compose up` ou `docker-compose down`).

Para aprendermos ainda mais sobre a ferramenta, a seguir veremos seus benefícios:

Facilidade de uso: simples de configurar e usar, especialmente útil para ambientes de desenvolvimento e teste.

Coesão: garante que todos os serviços necessários para uma aplicação estejam definidos e gerenciados em um único lugar.

Portabilidade: o arquivo `docker-compose.yml` pode ser compartilhado com outros membros da equipe, garantindo que todos possam replicar o ambiente de desenvolvimento local de forma consistente.

Com o Docker Compose, uma pessoa desenvolvedora pode criar um arquivo `docker-compose.yml` que define um serviço web, um banco de dados e um serviço de cache, todos executados como contêineres. Com um único comando, todos esses serviços são iniciados e configurados para funcionar juntos.

Integração entre ferramentas

Workflow típico de desenvolvimento e implantação

1. Desenvolvimento local com Docker Compose

Os(as) desenvolvedores(as) usam Docker Compose para configurar e testar microsserviços localmente. Cada serviço é definido em um arquivo `docker-`

`compose.yml`, facilitando o desenvolvimento colaborativo e garantindo consistência no ambiente de desenvolvimento.

2. Criação de imagens com Docker

Uma vez que o desenvolvimento e os testes locais estejam concluídos, as imagens Docker dos serviços são criadas utilizando arquivos `Dockerfile`. Elas são versionadas e armazenadas em um registro de contêineres, como Docker Hub ou um registro privado.

3. Orquestração com Kubernetes

Para ambientes de produção, as imagens Docker são implantadas em um cluster Kubernetes. Ele gerencia o ciclo de vida dos contêineres, garantindo que o número desejado de réplicas esteja em execução, distribuindo a carga de trabalho e lidando com falhas de forma automática.

4. Monitoramento e Logging

Ferramentas como Prometheus e Grafana são utilizadas para monitorar a performance dos contêineres e serviços. Logs são centralizados e analisados usando o ELK Stack (Elasticsearch, Logstash e Kibana) ou Fluentd, proporcionando visibilidade completa sobre o estado e a saúde dos microsserviços.

Práticas recomendadas para microsserviços

A integração e o deployment contínuo (CI/CD) são práticas essenciais no desenvolvimento de microsserviços. Elas garantem que o código é testado e implantado de forma automática, minimizando o tempo entre a escrita do código e sua disponibilização em produção. A adoção dessas práticas ajuda a manter a qualidade do código, reduzir o risco de erros em produção e aumentar a eficiência das equipes de desenvolvimento.

Já a Integração Contínua (CI) é o processo de automatizar a integração do código de todos os(as) desenvolvedores(as) de um projeto. Isso é feito por meio de builds e testes automáticos que são executados frequentemente, geralmente várias vezes ao dia. O objetivo principal é detectar e corrigir problemas o mais cedo possível no ciclo de desenvolvimento.

Ferramentas comuns (CI)

Jenkins: uma ferramenta de automação de código aberto que pode ser utilizada para automatizar todas as etapas do pipeline de integração contínua. Jenkins é altamente configurável e pode ser estendido com inúmeros plugins.

Travis CI: um serviço de CI baseado na nuvem que é fácil de configurar e integra-se bem com GitHub, amplamente utilizado em projetos de código aberto.

CircleCI: outra ferramenta de CI baseada na nuvem que se integra bem com sistemas de controle de versão como GitHub e Bitbucket. Ele oferece pipelines configuráveis e uma variedade de opções de execução.

Benefícios da CI

Detecção precoce de erros: os builds automáticos e testes contínuos ajudam a identificar problemas de integração e erros de código logo após a submissão.

Builds automáticos: automatizar o processo de build garante que cada mudança de código seja integrada e testada em um ambiente controlado.

Testes contínuos: a execução contínua de testes unitários, de integração e de sistema assegura que o código novo não quebre funcionalidades existentes.

Deploy Contínuo (CD)

O Deploy Contínuo (CD) vai um passo além da integração contínua. Ele garante que as mudanças no código que passam nos testes automáticos sejam implantadas automaticamente em um ambiente de produção. Isso acelera o ciclo de vida do desenvolvimento e permite que as equipes entreguem novas funcionalidades e correções de bugs de forma rápida e confiável.

Ferramentas comuns

Spinnaker: uma plataforma de entrega contínua multi-cloud que facilita a implantação rápida e segura de software, além de suportar várias ferramentas de orquestração de contêineres, como Kubernetes.

Argo CD: uma ferramenta de entrega contínua para Kubernetes que mantém aplicações Kubernetes sincronizadas com o estado desejado especificado em repositórios Git. Argo CD é particularmente útil em fluxos de trabalho GitOps.

Benefícios do CD

Automação do Deployment: automatiza o processo de implantação, reduzindo erros humanos e garantindo consistência entre os ambientes de desenvolvimento e produção.

Rollback automático: habilita rollback automático em caso de falhas, garantindo a estabilidade da aplicação em produção.

Deployment seguro e rápido: assegura que novas versões do software sejam implantadas de forma segura e eficiente, permitindo uma resposta rápida às mudanças nas necessidades do negócio.

Monitoramento e Logging

Monitorar e registrar logs de microsserviços é crucial para manter a visibilidade do sistema e diagnosticar problemas. O monitoramento contínuo e a análise de logs permitem detectar anomalias, identificar a causa raiz de problemas e melhorar o desempenho e a disponibilidade dos serviços.

Ferramentas de monitoramento

Prometheus: uma ferramenta de monitoramento e alerta de código aberto projetada para confiabilidade e escalabilidade. A Prometheus coleta métricas de tempo de execução de serviços e exibe dados em um painel visual.

Grafana: uma plataforma de análise e visualização de métricas que se integra bem com Prometheus e outras fontes de dados, permitindo a criação de dashboards personalizados para monitorar a saúde e o desempenho dos serviços.

Benefícios do monitoramento

Coleta de métricas: recolhe métricas detalhadas sobre a performance dos serviços, como tempo de resposta, uso de CPU e memória.

Monitoramento em tempo real: fornece visibilidade em tempo real sobre o estado dos serviços, permitindo a detecção e a resposta rápida a problemas.

Alertas personalizados: configura alertas personalizados para notificar automaticamente a equipe sobre anomalias ou falhas.

Ferramentas de Logging

ELK Stack (Elasticsearch, Logstash e Kibana): uma solução poderosa para coleta, análise e visualização de logs. Elasticsearch armazena e pesquisa logs, Logstash processa e transforma dados, e Kibana permite a visualização desses dados.

Fluentd: um coletor de logs de código aberto que unifica a coleta e o consumo de dados de logs. O Fluentd pode ser integrado com várias fontes de dados e destinos, facilitando a centralização dos logs.

Benefícios do Logging

Centralização de logs: consolida logs de diferentes serviços em um único lugar, facilitando a análise e a correlação de eventos.

Análise em tempo real: permite a análise de logs em tempo real, ajudando a identificar e solucionar problemas rapidamente.

Visualização de dados: ferramentas como Kibana proporcionam visualizações ricas e intuitivas dos dados de logs, facilitando o entendimento do comportamento do sistema.

Maiores diferenças

Comparação de desenvolvimento entre microserviços e monolitos

- **Desenvolvimento**

- **Monolitos:** desenvolvimento centralizado, uma única base de código. É mais fácil começar com um monolito, pois todos os componentes estão em um único repositório, facilitando a integração inicial. No entanto, à medida que a aplicação cresce, pode ser difícil gerenciar e escalar a base de código.
- **Microserviços:** desenvolvimento descentralizado, múltiplas bases de código. Cada serviço pode ser desenvolvido e implantado de forma independente, o que facilita a escalabilidade e a manutenção a longo prazo, mas aumenta a complexidade inicial do projeto.

- **Teste**

- **Monolitos:** testes mais simples devido à integração em uma única base de código. Todos os componentes estão disponíveis localmente, facilitando a execução de testes end-to-end.
- **Microserviços:** testes mais complexos, necessitando de mocks e simulações de comunicação entre serviços. Testar microserviços exige uma abordagem robusta para simular interações entre serviços e garantir a compatibilidade.

- **Deploy**

- **Monolitos:** deployment único, mais simples de gerenciar, porém mais arriscado, já que qualquer mudança requer a reimplantação da aplicação inteira, aumentando o risco de falhas.
- **Microserviços:** deployment independente, mais seguro, mas mais complexo de orquestrar. Cada serviço pode ser implantado separadamente, permitindo atualizações frequentes e isoladas, mas requer uma infraestrutura robusta de CI/CD.

- **Escalabilidade:**

- **Monolitos:** a escalabilidade é limitada pela necessidade de replicar a aplicação completa, o que pode ser ineficiente.
- **Microserviços:** escalabilidade granular, permitindo escalar apenas os serviços necessários. Logo, cada serviço pode ser escalado independentemente com base nas suas necessidades específicas, tornando a utilização de recursos mais eficiente.

MERCADO, CASES E TENDÊNCIAS

O desenvolvimento de microserviços tem sido amplamente adotado em várias indústrias, destacando-se como uma abordagem eficaz para melhorar a escalabilidade, a flexibilidade e a resiliência dos sistemas. A seguir veremos casos de uso reais e tendências emergentes que ilustram como empresas de diferentes setores estão aproveitando essa arquitetura para resolver problemas complexos e alcançar novos níveis de eficiência.

Utilização de microserviços em desenvolvimento de software

1. Setor de tecnologia

Empresas de tecnologia foram as pioneiras na adoção de microserviços devido à necessidade de escalar rapidamente e inovar constantemente. Gigantes da tecnologia, como Netflix, Uber e Spotify, foram as primeiras companhias a adotar e popularizar essa arquitetura.

A Netflix, por exemplo, migrou de uma arquitetura monolítica para microserviços para lidar com o aumento de usuários e a necessidade de implantar novas funcionalidades de forma ágil.

2. Setor financeiro

Bancos e instituições financeiras estão adotando microserviços para modernizar suas infraestruturas legadas e atender às demandas por serviços digitais mais rápidos e seguros.

O Goldman Sachs, grupo financeiro multinacional que oferece uma ampla gama de serviços financeiros, utiliza microserviços para melhorar a performance e a escalabilidade de suas aplicações financeiras, permitindo uma resposta mais rápida às mudanças do mercado.

3. E-commerce

Plataformas de e-commerce, como Amazon e Shopify, utilizam microserviços para gerenciar suas operações de vendas, desde o catálogo de produtos até o processamento de pagamentos e logística.

Principais diferenças ao desenvolver microserviços

A Shopify é um exemplo disso, pois nessa empresa há a utilização de Kubernetes para orquestrar seus microserviços, garantindo que suas operações de e-commerce permaneçam escaláveis e resilientes durante picos de demanda, como a Black Friday.

Case de uso: Uber

A Uber adotou uma arquitetura de microserviços para suportar seu rápido crescimento global. Com a expansão em diferentes mercados e a diversificação de serviços (como Uber Eats e Uber Freight), a necessidade de uma arquitetura escalável e flexível tornou-se evidente.

Benefícios

Escalabilidade global: poder escalar seus serviços para diferentes regiões do mundo, atendendo a uma base de usuários diversificada.

Flexibilidade: a arquitetura de microserviços permite que a Uber adapte rapidamente seus serviços às necessidades locais e regulatórias.

Resiliência: a separação dos serviços melhora a resiliência geral do sistema, permitindo que a Uber mantenha a disponibilidade mesmo durante picos de demanda.

Tendências emergentes

Arquitetura de microserviços orientada a domínios (DOMA)

A Arquitetura de Microserviços Orientada a Domínios (DOMA) é uma abordagem que agrupa microserviços em torno de domínios de negócios específicos. Essa tendência está ganhando popularidade, pois ajuda a alinhar melhor a arquitetura técnica com os objetivos de negócios. Seus benefícios incluem alinhamento com negócios, pois melhora a colaboração entre equipes de negócios e tecnologia, facilitando a entrega de valor ao cliente, e escalabilidade e flexibilidade, já que cada domínio pode ser escalado e desenvolvido de forma independente, aumentando a flexibilidade e a eficiência.

Observabilidade e monitoramento avançado

Com a crescente complexidade das arquiteturas de microserviços, a observabilidade tornou-se uma prioridade. Ferramentas avançadas de

Principais diferenças ao desenvolver microserviços

monitoramento, logging e tracing estão sendo adotadas para garantir que os sistemas permaneçam estáveis e performáticos. As ferramentas utilizadas para isso são Prometheus e Grafana para monitoramento de métricas e visualização, e Jaeger e Zipkin para tracing distribuído, permitindo rastrear solicitações por meio de múltiplos serviços.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos as ferramentas e tecnologias essenciais para o desenvolvimento de microsserviços. Discutimos a flexibilidade na escolha de linguagens de programação e frameworks, destacando como a escolha deve considerar as necessidades do projeto, a experiência da equipe e o ecossistema de ferramentas disponíveis. Analisamos detalhadamente a importância da containerização e da orquestração, com foco em ferramentas como Docker e Kubernetes, que garantem a consistência do ambiente, a escalabilidade e a gestão eficiente de recursos.

Também abordamos práticas recomendadas para desenvolvimento, teste e deploy contínuo (CI/CD), enfatizando a necessidade de integração contínua para detecção precoce de erros e deployment contínuo para automação segura e eficiente das implantações. Além disso, discutimos a importância do monitoramento e logging para manter a visibilidade do sistema e diagnosticar problemas de forma proativa.

Por fim, comparamos o desenvolvimento de microsserviços com o de monolitos, destacando as principais diferenças em termos de desenvolvimento, testes, deployment e escalabilidade, analisamos tendências emergentes no mercado, vimos um caso de uso real e aprendemos sobre mais sobre a adoção de práticas avançadas, como a arquitetura de microsserviços orientada a domínios e a observabilidade aprimorada.

REFERÊNCIAS

FOWLER, M; LEWIS, J. Microservices: A definition of this new architectural term. Martin Fowler, 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 24 set. 2024.

INTRODUCING Domain-Oriented Microservice Architecture. **Uber**, 2020. Disponível em: <https://www.uber.com/en-BR/blog/microservice-architecture/>. Acesso em: 24 set. 2024.

NEWMAN, S. **Building Microservices**. 2nd ed. [s.l.]: O'Reilly Media, 2021.

RICHARDSON, C. **Microservices Patterns: With examples in Java**. [s.l.]: Manning Publications, 2018.

Principais diferenças ao desenvolver microserviços

PALAVRAS-CHAVE

Arquiteto, Arquitetura de Software, Microserviços, Monolito, Logging, CI, CD, Monitoramento, Linguagens.

EXEMPLO

POSTECH